

ACAN2518FD — Supplement

Sleep, wake-up, and what changed from ACAN2517FD

Library version 1.0.0 · Supplement to *ACAN2517FD Arduino library 2.1.16* by Pierre Molinaro

Contents

- | | |
|--|--|
| 1. About this document | 6. Parking the transceiver |
| 2. What this fork is | 7. The SPI clock ceiling |
| 3. Why the sleep support had to be added | 8. Reading Pierre's manual with this library |
| 4. The sleep/wake API | 9. Status |
| 5. Putting it together | 10. Licence and credit |

1. About this document

ACAN2518FD is a fork of Pierre Molinaro's **ACAN2517FD 2.1.16** (MIT). His 61-page manual — `acan2517FD.pdf`, in this same `extras/` directory — remains the reference for the driver. It describes the data flow, the bit-timing solver, the message classes, the filters, the FIFOs, the clock configuration and every existing method, and **all of it still applies**.

This supplement covers only the delta: the naming, the sleep/wake API that is new here, and how to read his manual alongside this library.

Reading the two together

Everything in `acan2517FD.pdf` is correct for this library after four substitutions:

In Pierre's manual	In this library
<code>ACAN2517FD</code>	<code>ACAN2518FD</code>
<code>ACAN2517FDSettings</code>	<code>ACAN2518FDSettings</code>
<code>ACAN2517FDFilters</code>	<code>ACAN2518FDFilters</code>
<code>#include <ACAN2517FD.h></code>	<code>#include <ACAN2518FD.h></code>

`CANFDMessage`, `CANMessage` and `ACANFDBuffer` are **unchanged** — they are shared across the whole ACAN family and renaming them would have broken compatibility with code written for any of those drivers. It also means this library and ACAN2517FD cannot both be included in one sketch: each defines those classes.

Two sections of his manual need a note beyond the rename, and they are in §8 below.

2. What this fork is

Derived from	ACAN2517FD 2.1.16, Pierre Molinaro
Upstream	https://github.com/pierremolinaro/acan2517FD
This fork	https://github.com/nemirogl/acan2518FD
Licence	MIT — retained, with a copyright line added for the fork's changes
Target	MCP2518FD only. Upstream also serves the MCP2517FD and the MCP251863

Everything that differs

1. **Renamed**, as in §1.
2. **Scope narrowed** to the MCP2518FD.
3. **The SPI clock ceiling stated explicitly** — §7.
4. **A sleep/wake API** — §4 onward, and the reason the fork exists.

Nothing else was touched. No existing method changed behaviour.

An honest note on "specialised for the MCP2518FD"

There is nothing in upstream's *code* that distinguishes the two parts: same register map, same oscillator options, same GPIO handling, same RAM size, no conditional compilation anywhere. A reader expecting a porting effort will not find one.

The differences that matter are real, but they are in Pierre's **documentation**, not in his code — and he is unambiguous about them. His §3, *MCP2517FD or MCP2518FD?*, opens:

"In short: I recommend using a MCP2518FD. My opinion is that the MCP2517FD has hardware bugs."

and goes on to document unreliable software reset on the MCP2517FD, the `nCS` pull-up he considers necessary there and not on the MCP2518FD, and Restricted Operation Mode entered above 1 Mbit/s. **That section is worth reading in full**, and it is the honest justification for a 2518-only fork: not that the code needed changing, but that the part is the one to build on.

3. Why the sleep support had to be added

`ACAN2517FD::begin()` configures the controller's interrupts like this (`ACAN2517FD.cpp`, lines 451–456 in 2.1.16):

```
data8 = (1 << 1) ; // Receive FIFO Interrupt Enable
data8 |= (1 << 0) ; // Transmit FIFO Interrupt Enable
writeRegister8 (INT_REGISTER + 2, data8) ;
data8 = (1 << 2) ; // TXATIE
writeRegister8 (INT_REGISTER + 3, data8) ;
```

CiINT (0x01C) carries its interrupt flags in bits 0-15 and its enables in bits 16-31. The code above sets **TXIE** (bit 16), **RXIE** (bit 17) and **TXATIE** (bit 26). **WAKIE — bit 30 — is never set.**

The consequence is quiet and complete. A controller put into Sleep mode through the public API *does* wake on a dominant edge at RXCAN — that transition is automatic in silicon, and the device moves itself to Configuration mode and raises **WAKIF**. But with WAKIE clear it **asserts nothing on its INT pin**. A host microcontroller sleeping with that pin armed as its wake source sleeps straight through the event.

There was no way to correct this from outside the library. `writeRegister8()` and its siblings are `private`, and there is no wake-related call in the public API. The choice was to patch an installed copy — which silently makes a build unreproducible for anyone else — or to fork. This is the fork.

4. The sleep/wake API

Seven public methods. They are additions; nothing existing was modified.

4.1 `maskInterruptsForSleep`

```
void maskInterruptsForSleep (void) ;
```

Sets **WAKIE**, clears **RXIE**, **TXIE** and **TXATIE**, clears all pending interrupt flags, and saves the previous enable bytes for `restoreInterruptsAfterSleep()`.

This is the method the fork exists for, and it does two jobs.

Setting WAKIE is the fix described in §3: without it a wake-up asserts nothing.

Clearing the other three is the mirror image of the same problem, and it is easy to overlook. The INT pin is active **LOW**, and a host arming a level-triggered wake arms it on LOW. An INT already held low by an ordinary receive or transmit condition at the moment the host sleeps is an *instant* wake — and then a sleep/wake loop that draws more current than simply staying awake would have. After this call, while the controller is asleep, the only event that can pull INT low is a wake-up.

4.2 `restoreInterruptsAfterSleep`

```
void restoreInterruptsAfterSleep (void) ;
```

Clears the pending flags and writes back the enable bytes saved by `maskInterruptsForSleep()`, so that polled or interrupt-driven operation behaves exactly as it did before the sleep. Does nothing if the mask was never applied, so it is safe on any unwind path.

4.3 `setWakeUpFilter`

```
bool setWakeUpFilter (const bool inEnabled, const uint8_t inFilterTime = 0) ;
```

Configures `CiCON.WAKFIL` (bit 8) and `CiCON.WFT` (bits 9-10), the controller's wake-up filter, which rejects glitches on RXCAN so that electrical noise on a parked bus does not rouse the system. `inFilterTime` is WFT, 0-3; longer is more immune and less sensitive.

Must be called in Configuration mode — several CiCON bits are writable only there.

The write is **read back**, and the return value reports what the device actually has rather than what it was asked for. A filter that silently failed to arm is worth knowing about on the bench rather than assuming.

Not yet characterised. *How WFT's filter times relate to bit times is not something this fork has measured, and a long filter on a slow bus is the obvious way to build something that never wakes. Start at 0, or with the filter off, and measure.*

4.4 `setOperationModeAndWait`

```
bool setOperationModeAndWait (const ACAN2518FDSettings::OperationMode inMode,  
                             const uint32_t inTimeoutMs = 20) ;
```

Requests an operating mode and polls `CiCON.OPMOD` until it matches. Returns `false` on timeout.

The bounded wait is the point. Pierre's `setOperationMode()` (his §22.2) requests the mode and returns immediately; whether the controller reached it is the caller's problem. Mode changes on this chip can wait on the bus, so "requested" and "reached" are genuinely different states and the sleep path has to know which one it is in.

4.5 `enterSleepMode`

```
bool enterSleepMode (const uint32_t inTimeoutMs = 20) ;
```

Requests Sleep and verifies it, accepting **either** witness:

- `CiCON.OPMOD` reads Sleep, or
- `OSC.OSCDIS` is set — the oscillator has stopped.

Either is sufficient, deliberately. In Sleep the oscillator is stopped, and how OPMOD reads back through a stopped clock is not something worth staking a power design on. A false negative here means a system that never sleeps at all, which is a worse outcome than a slightly loose check. `OSCDIS` is the independent confirmation, and it is the same bit Pierre's `performSleepModeToConfigurationMode()` uses to decide whether the device was asleep.

Returns `false` only if neither appears within the timeout — at which point the caller should stay awake rather than press on.

4.6 `isAsleep`

```
bool isAsleep (void) ;
```

True when `OSC.OSCDIS` is set, i.e. the oscillator is stopped and the controller is in Sleep.

4.7 `wakeUpFromSleepMode`

```
bool wakeUpFromSleepMode (const uint32_t inTimeoutMs = 5) ;
```

Clears `OSCDIS` and waits, **bounded**, for `OSCRDY`. Returns true when the oscillator is running.

It is close in purpose to Pierre's `performSleepModeToConfigurationMode()` (his §22.4), and it exists alongside it for two reasons.

It cannot hang. Upstream's readiness loop is

```
while (wait) { wait = (readRegister8 (OSC_REGISTER + 1) & (1 << 2)) == 0 ; }
```

which never terminates against an absent or unresponsive controller.

It is safe to call before `begin()` — which is exactly when it is needed. See §5.2.

5. Putting it together

5.1 Going to sleep

The order is the design. Every step that can refuse, refuses *before* the system has given up the ability to say so.

```

//--- 1. Drain. A frame left in the RX FIFO holds RXIF, and RXIF holds INT low,
//      which would wake the host the instant it slept.
CANFDMessage frame ;
while (can.receive (frame)) {}

//--- 2. From here, only a wake-up may assert INT.
can.maskInterruptsForSleep () ;

//--- 3. Configuration mode FIRST. It is always reachable, it never waits on bus
//      idle, and it is the only mode in which the CiCON configuration bits are
//      reliably writable. Requesting Sleep straight from Normal means asking for
//      a mode change while the transceiver may already be holding RXCAN
//      dominant - and mode changes are the one thing on this chip that wait on
//      the bus.
if (!can.setOperationModeAndWait (ACAN2518FDSettings::Configuration, 20)) {
    /* stay awake */
}

//--- 4. Optional: reject glitches on a parked bus.
const bool filterArmed = can.setWakeUpFilter (true, 0) ;

//--- 5. Park the transceiver, if your board can - see §6.

//--- 6. Sleep, verified.
if (!can.enterSleepMode (20)) {
    /* stay awake - see below */
}

```

Then confirm the INT pin actually reads HIGH before arming a level-triggered wake, and only then sleep the host.

Every failure path should unwind and stay awake. An awake controller works; a half-slept one is either unreachable or in a wake loop. Unwinding is the reverse of the sequence: un-park the transceiver, re-request the operating mode, and call `restoreInterruptsAfterSleep()`.

5.2 Coming back

After a host deep sleep — where the host resets — there is a trap worth stating plainly:

A host MCU reset does not reset the controller. *There is no reset line between them. Every controller is still in Sleep with its oscillator stopped, and `begin()` opens by requesting a mode change and issuing a chip RESET against that stopped oscillator.*

So restart the oscillator first:

```
SPI.begin (...) ;  
can.wakeUpFromSleepMode () ;    // BEFORE begin(). Harmless if already awake:  
can.begin (settings, ...) ;    // OSCDIS reads clear and nothing is written.
```

After a host light sleep, where RAM and driver state survive, resume in place instead of re-initialising:

```
can.wakeUpFromSleepMode () ;  
// ... un-park the transceiver ...  
can.setOperationModeAndWait (yourNormalMode, 20) ;  
can.restoreInterruptsAfterSleep () ;
```

A transceiver that has just come out of standby may still be holding RXCAN dominant with a latched wake event, and the controller cannot reach *any* operating mode until it lets go. One retry, after driving the PHY to Normal again, is worth having.

5.3 The INT-quiet trap, stated once more

It is the single most likely way to get this wrong, so:

Never arm a LOW-level wake while an armed INT pin is already LOW. It wakes instantly, every time, and the resulting loop draws more than staying awake. Drain the FIFOs, mask the interrupts, then *verify* the pin reads HIGH — with a timeout, and a decision to stay awake if it does not.

6. Parking the transceiver

Sleeping the controller is only half of a low-power bus node. The transceiver has to go into its own low-power mode too, with its wake receiver still watching the bus — that is what produces the dominant edge at RXCAN that wakes the controller.

On boards that route the transceiver's standby pin to the controller's GPIO0, this is Pierre's §21 API. With GPIO0 wired straight to an active-HIGH standby input:

```
can.gpioWrite (0, HIGH) ;        // park  
can.gpioSetMode (0, OUTPUT) ;
```

and the reverse to bring it back. Note the latch-then-direction order: writing the output latch while the pin is still an input means the pin takes the intended level the instant it becomes an output, with no transient through the post-reset default.

`configureGPIO0AsXSTBY()` (his §21.4) hands the same job to the controller's automatic standby feature instead. Driving the pin explicitly is the more predictable of the two.

Park the transceiver *after* the controller has left Normal mode, not before — a parked PHY holding RXCAN dominant is exactly what blocks a mode change.

7. The SPI clock ceiling

Upstream computes the full-speed SPI clock as `0.4 × SYSCLK` and applies it unclamped. This fork adds:

```
const uint32_t MCP2518FD_MAX_SPI_CLOCK = 20UL * 1000UL * 1000UL ; // DS20006027
```

At every oscillator the part supports, `0.4 × SYSCLK` is already below that — 16 MHz with a 40 MHz oscillator — so **this changes no behaviour**. It is there so the device limit lives in the code rather than emerging from an arithmetic expression, and so that a later change to the ratio cannot quietly walk past it.

8. Reading Pierre's manual with this library

Apply the rename from §1 throughout. Two sections need more than that.

§3 — *MCP2517FD or MCP2518FD?*

Still worth reading, and now settled: this library drives the MCP2518FD only. His observations about MCP2517FD reset reliability, the `nCS` pull-up, and Restricted Operation Mode above 1 Mbit/s describe the part this fork deliberately does not target.

§22.4 — *The performSleepModeToConfigurationMode method*

Unchanged and still present. `wakeUpFromSleepMode()` (§4.7 here) does the same job with a bounded wait and is usable before `begin()`; prefer it in new code.

Everything else — the data flow, `CANFDMessage`, the wiring chapters, clock configuration, the FIFOs, payload sizes, RAM usage, `tryToSend`, reception and filters, the settings class, the GPIO methods, the error counters — applies as written.

9. Status

The sleep/wake path has not been confirmed on hardware.

It is derived from the MCP2518FD datasheet (DS20006027) and the register map Pierre documents from DS20005688B, it has been reviewed carefully, and it compiles — but at the time of writing nobody has measured it on a board. Treat this part of the library as a considered proposal rather than a proven one, and check the repository for a newer statement before relying on it.

The rest of the library is upstream 2.1.16, which has been in the field for years.

10. Licence and credit

MIT. The copyright is shared: **Pierre Molinaro** for the original work and for everything in this library that is still his — which is nearly all of it — and the fork's own changes on top. See `LICENSE`.

`acan2517FD.pdf` in this directory is **Pierre's document**, reproduced unmodified. It is his writing and his API documentation, and it deserves to be read as such.

If the sleep/wake support is useful, it belongs upstream rather than in a fork. Anyone who finds it so is encouraged to say so on `https://github.com/pierremolinaro/acan2517FD` — if it lands there, this fork can retire.